

```
// Version number functions
/////////////////////////////////////////////////////////////////
/// \name Configuration
/// \brief The following methods give the version number of the driver, which also match
that of the installer.
```

```
/// @{
MS_DECL_SPEC unsigned int getVersionMajor();
MS_DECL_SPEC unsigned int getVersionMinor();
MS_DECL_SPEC unsigned int getVersionPatch();
/// @}
```

```
// General library functions
/////////////////////////////////////////////////////////////////
/// \name Configuration
/// \brief The following methods relate to the configuration of the driver and the creation
and management of PF32 objects.
```

```
/// @{
```

```
/** \brief Destroy all instantiated PF32 objects, unload library and close log. Use this
* method if quitting after a critical error. Any pointers to PF32 objects in the client
code
```

```
* will become invalid as the objects they point to will be deleted by this method.
*/
```

```
MS_DECL_SPEC void closeAll();
```

```
/** \brief Find out how many PF32 objects are currently instantiated.
*/
```

```
MS_DECL_SPEC int noOfPF32sInstantiated();
```

```
/** \brief Returns a specific instance of PF32 objects by its position in the queue.
* This is determined by the order in which objects have been instantiated and
deleted.
```

```
* \param index Index number of the PF32 object to be returned.
* \return Handle to specified PF32 object instance.
*/
```

```
MS_DECL_SPEC PF32_HANDLE getPF32InstanceByIndex(int index);
```

```
// Logging
```

```

/** \brief Set the minimum severity level of log messages to be written to log file.
 * In Windpows, the log file can be found in the temp directory. Open a file explorer and
type %temp%
 * in the address bar for its location. File name is PF32_YYYY_MM_DD.log
 * \param newLogLevel New minimum severity level of log messages to be written to
log file. Default setting is info messages upwards.
 */
MS_DECL_SPEC void setLogFileLevel(int newLogLevel);

```

```

/** \brief Query the current minimum verbosity level of log messages to be written to log
file.
 * \return Current minimum verbosity level of log messages to be written to log file.
 */
MS_DECL_SPEC int getLogFileLevel();

```

```

/** \brief Set the minimum severity level of log messages to be written to standard
output stream.
 * \param newLogLevel New minimum severity level of log messages to be written to
standard output stream. Default setting is off.
 */
MS_DECL_SPEC void setLogStreamLevel(int newLogLevel);

```

```

/** \brief Query the current minimum severity level of log messages to be written to
standard output stream.
 * \return Current minimum verbosity level of log messages to be written to standard
output stream.
 */
MS_DECL_SPEC int getLogStreamLevel();

```

```

/** \brief Set the minimum severity level of log messages to be saved to memory. These
can be accessed programmatically.
 * Default setting is error.
 * \param newLogLevel New minimum severity level of log messages to be saved to
memory. Default setting is error messages only.
 */
MS_DECL_SPEC void setLogMemoryLevel(int newLogLevel);

```

```

/** \brief Query the current minimum severity level of log messages to be saved to
memory.
 * \return Current minimum verbosity level of log messages to be written to saved to
memory.
 */

```

```
MS_DECL_SPEC int getLogMemoryLevel();
```

```
/** \brief Get log messages saved in memory. These are the same messages saved to the  
log file and output to stream
```

```
    * although all three options can be set to output log messages of different severity.
```

```
    * \param noOfMessages Pointer to an integer that can store the number of messages.
```

```
The array of strings is
```

```
    null terminated so you may not need this, in which case you can pass in  
NULL instead.
```

```
    But it can be useful if you need to know in advance how many messages  
there are.
```

```
    \return An array of strings that is null terminated. Each string is also null terminated.
```

```
*/
```

```
MS_DECL_SPEC char ** getLogMessages(unsigned int * noOfMessages);
```

```
/** \brief Constructor to create and initialise a PF32 comms object and return a handle to  
it.
```

```
    * \return Pointer to a new PF32 object.
```

```
*/
```

```
MS_DECL_SPEC PF32_HANDLE PF32_construct();
```

```
/** \brief Constructor to create and initialise a PF32 comms object and return a handle to  
it,
```

```
    * taking a custom camera firmware file path as an argument.
```

```
    * Only call this method if you have been provided with a application specific custom  
firmware file by Photon Force.
```

```
    * This provides an alternative to using the standard \ref PF32_construct method and  
then calling \ref loadCustomFirmware.
```

```
    * \param firmwareFileName Absolute or relative path to the custom camera firmware  
file to be loaded.
```

```
    * \return Handle to new PF32 object.
```

```
*/
```

```
MS_DECL_SPEC PF32_HANDLE PF32_constructWithCustomFirmware(const char *  
firmwareFileName);
```

```
/** \brief Destructor to close a PF32 comms object.
```

```
    * It is essential that no further calls are made using this handle once the destructor has  
be called.
```

```
    * \param pf32 Handle to the PF32 object to be destroyed.
```

```
*/
```

```
MS_DECL_SPEC void PF32_destruct(PF32_HANDLE pf32);
```

/** \brief This method is provided solely for the benefit of customers who have requested application specific camera firmware from Photon Force.

- * It is used to load a custom camera firmware file. If successful, this method will configure the camera return the connection status as ready.

- * If an error occurs, the method will return the connection status as error.

- * Alternatively, you may wish to use the \ref PF32_constructWithCustomFirmware method in place of \ref PF32_construct to create a PF32 object

- * which will automatically load the specified custom firmware path when the camera is connected.

- * If you have not obtained custom camera firmware from Photon Force, both of these methods may be ignored,

- * as the default camera firmware is loaded automatically.

- * \param pf32 Handle to the PF32 object.

- * \param firmwareFileName Absolute or relative path to the custom camera firmware file to be loaded.

- * \return Connection status. If successful, this method will configure the camera return the connection status as ready.

- * If an error occurs, the method will return the connection status as error.

*/

MS_DECL_SPEC PF32_conn_status loadCustomFirmware(PF32_HANDLE pf32, const char *firmwareFileName);

/// @}

/** \brief Returns number of columns, or the width of the sensor array.

- * \param pf32 Handle to the PF32 object to be queried.

- * \return Number of columns, or the width of the sensor array.

*/

MS_DECL_SPEC unsigned int getWidth(PF32_HANDLE pf32);

/** \brief Returns number of rows, or the height of the sensor array.

- * \param pf32 Handle to the PF32 object to be queried.

- * \return Number of rows, or the height of the sensor array.

*/

MS_DECL_SPEC unsigned int getHeight(PF32_HANDLE pf32);

/** \brief Returns number of pixels in the sensor array, calculated as getWidth() * getHeight().

- * \param pf32 Handle to the PF32 object to be queried.

- * \return Number of pixels in the sensor array, calculated as getWidth() * getHeight().

*/

MS_DECL_SPEC unsigned int getNoOfPixels(PF32_HANDLE pf32);

/** \brief Returns number of pixels in the sensor array that are currently enabled if setRegionsOfInterest() has switched some off.

* The number of columns (width) is not affected by setting regions of interest as values are substituted with zero. But Roi does

* affect the number of rows (height), so this value will be getWidth() * getEnabledHeight().

* \param pf32 Handle to the PF32 object to be queried.

* \return Number of pixels in the sensor array, calculated as getWidth() * getEnabledHeight().

*/

MS_DECL_SPEC unsigned int getEnabledNoOfPixels(PF32_HANDLE pf32);

/** \brief Returns number of rows that are currently enabled if setRegionsOfInterest() has switched some off.

There is no getEnabledWidth() because the number of columns does not change with Regions of Interest.

Instead values are substituted with zero for the columns that are switched off.

* \param pf32 Handle to the PF32 object to be queried.

* \return Number of pixels in the sensor array, calculated as getWidth() * getHeight().

*/

MS_DECL_SPEC unsigned int getEnabledHeight(PF32_HANDLE pf32);

/** \brief Returns number of Time to Digital Converter (TDC) codes.

* \param pf32 Handle to the PF32 object to be queried.

* \return Number of Time to Digital Converter (TDC) codes.

*/

MS_DECL_SPEC unsigned int getNoOfTDCCodes(PF32_HANDLE pf32);

/** \brief Method to query system link status.

* \param pf32 Handle to the PF32 object to be queried.

*/

MS_DECL_SPEC PF32_conn_status getLinkStatus(PF32_HANDLE pf32);

// Operation

/** \brief Set PF32 sensor acquisition mode.

* \param pf32 Handle to the PF32 object to be updated.

* \param mode Mode to set the PF32 camera to.

*/

```

MS_DECL_SPEC void setMode(PF32_HANDLE pf32, PF32_mode mode);

/////////////////////////////////////////////////////////////////
/// \name I2C comms methods
/// \brief The I2C methods allow the sensor's I2C registers to be read from (getI2C) or
written to (writeI2C).
/// Note that the \ref setMode function provides the simplest means of reconfiguring
the PF32 camera.
/// It is not expected that the user would need to manually change I2C register settings in
all but the most advanced use-cases.
/// @{

/** \brief Writes to the specified sensor I2C register.
 * \param pf32 Handle to the PF32 object to be updated.
 * \param data New value for the specified register.
 * \param address Sensor I2C register address to be updated.
 * \return Boolean value indicating success or failure.
 */
MS_DECL_SPEC bool setI2C(PF32_HANDLE pf32, int data, int address);

/** \brief Reads from the specified sensor I2C register.
 * \param pf32 Handle to the PF32 object to be queried.
 * \param address Sensor I2C register address to be queried.
 * \param dataOut Pointer to integer to which to write the returned I2C register
contents.
 * \return Boolean value indicating success or failure.
 */
MS_DECL_SPEC bool getI2C(PF32_HANDLE pf32, int address, int* dataOut);
/// @}

/////////////////////////////////////////////////////////////////
/// \name DAC comms methods
/// \brief The Digital to Analog Converter (DAC) methods allow the PF32 bias voltage
settings to be read (\ref getDAC), updated (\ref setDAC), or restored to defaults (\ref
applyDACDefaultValues).
/// Additionally, the maximum permitted values may be queried (\ref getMaxValueOfDAC).
The minimum value for all DACs is 0. All DAC values are set or returned as integer values specified
in mV.
/// Note that the VBD DAC applies a hardware gain of -4, i.e. a setting of 3700
corresponds to -14.8V.
/// The \ref getDAC, \ref setDAC, and \ref getMaxValueOfDAC methods address individual
DACs, and therefore expect the target DAC type to be specified as a \ref PF32_DAC_type enum.
/// The \ref applyDACDefaultValues method resets all DACs to their default values, and so
does not take a DAC type input.

```

```

/// @{

/** \brief Instruct the PF32 to immediately reset all of its DACs to their default values.
 *   The updated values may be read by calling \ref getDAC.
 *   \param pf32 Handle to the PF32 object to be updated.
 */
MS_DECL_SPEC void applyDACDefaultValues(PF32_HANDLE pf32);

/** \brief Program the specified DAC to value provided, specified as an integer in mV.
 *   Note that the VBD DAC applies a hardware gain of -4, i.e. to set -14.8V, the command
would be \ref setDAC(hnd, \ref VEB_DAC, 3700);
 *   \param pf32 Handle to the PF32 object to be updated.
 *   \param dacType DAC to be updated.
 *   \param value New value to be written to the specified DAC, as an integer in mV.
 *   \return Boolean value indicating success or failure.
 */
MS_DECL_SPEC bool setDAC(PF32_HANDLE pf32, PF32_DAC_type dacType, int value);

/** \brief Query the specified DAC value, returned as an integer in mV.
 *   Note that the VBD DAC applies a hardware gain of -4, i.e. when set to -14.8V, the
command \ref getDAC(hnd, \ref VEB_DAC); would return 3700.
 *   \param pf32 Handle to the PF32 object to be queried.
 *   \param dacType DAC to be queried.
 *   \return Value returned from the specified DAC, as an integer in mV.
 */
MS_DECL_SPEC int getDAC(PF32_HANDLE pf32, PF32_DAC_type dacType);

/** \brief Query the maximum permitted value which may be programmed via \ref
setDAC for the specified DAC, returned as an integer in mV.
 *   \param pf32 Handle to the PF32 object to be queried.
 *   \param dacType DAC to be queried.
 *   \return Maximum permitted value for the specified DAC, as an integer in mV.
 */
MS_DECL_SPEC int getMaxValueOfDAC(PF32_HANDLE pf32, PF32_DAC_type dacType);
/// @}

////////////////////////////////////
/// \name Streaming
/// \brief The streaming methods relate to data transfer from the PF32 camera.
/// @{

/** \brief Method to read frames from the camera when the camera is using firmware
that returns 8-bit data.
 *   \param pf32 Handle to the PF32 object to read from.

```

- * \param data Pointer to buffer in which to write the captured frame data. The buffer must be at least noOfFrames * \ref getEnabledNoOfPixels uint8 bytes.
- * \param noOfFrames Number of frames to read. The minimum is 0.
- * \param buffered When false, an unbuffered read is performed (live data is captured directly from the camera.) When true, a buffered read is performed (data is returned from a software circular buffer, which is filled by a background thread.)
- * \param performInitialPurge If true then the circular buffer (only for buffered reads) and the firmware FIFO will be purged before reading commences.
- * \return Boolean value indicating success or failure.
- */

```
MS_DECL_SPEC bool getNextFrames(PF32_HANDLE pf32, uint8_t *data, unsigned int noOfFrames, bool buffered, bool performInitialPurge);
```

```
/** \brief Use this method for third party languages that cannot cast pointers.
```

- * \param pf32 Handle to the PF32 object to read from.
- * \param data Pointer to buffer in which to write the captured frame data. The buffer must be at least noOfFrames * \ref getEnabledNoOfPixels uint16 words.
- * \param noOfFrames Number of frames to read. The minimum is 0.
- * \param buffered When false, an unbuffered read is performed (live data is captured directly from the camera.) When true, a buffered read is performed (data is returned from a software circular buffer, which is filled by a background thread.)
- * \param performInitialPurge If true then the circular buffer (only for buffered reads) and the firmware FIFO will be purged before reading commences.
- * \return Boolean value indicating success or failure.
- */

```
MS_DECL_SPEC bool getNextFrames_short(PF32_HANDLE pf32, uint16_t *data, unsigned int noOfFrames, bool buffered, bool performInitialPurge);
```

```
/** \brief Some third party languages do not provide easy or efficient ways to perform multi-threading.
```

This method spawns each camera in a new thread calling getNextFrames() in parallel.

You can use this method instead instead of creating and destroying your own sessions.

- * \param noOfCameras How many threads to spawn off. The method expects handles, data and noOfFrames to each be arrays of this length.
- * \param handles Array of PF32_HANDLES to read from.
- * \param data Array of pointers to buffer in which to write the captured frame data. The buffer must be at least noOfFrames * \ref getEnabledNoOfPixels uint8 bytes.
- * \param noOfFrames Array of number of frames to read. The minimum is 0.
- * \param performInitialPurge If true then the firmware FIFO will be purged before reading commences.


```

*/
MS_DECL_SPEC void getNextFramesParallel(unsigned int noOfCameras, PF32_HANDLE *
handles, uint8_t ** data, unsigned int * noOfFrames, bool buffered, bool performInitialPurge);

```

/** \brief Sessions run multiple cameras in parallel, each camera in its own thread. This saves the client code from having to use their own multi-threading,

which may not be easily possible with some third party languages. If you want this functionality but don't want to use sessions then you can call

getNextFramesParallel() instead which will do it for you.

This method will create a new session.

\return A handle that can be passed to other methods to identify the session.

```

*/
MS_DECL_SPEC SESSION_HANDLE createSession();

```

/** \brief Add a camera to the session. This will be run in its own separate thread when executeSession() gets called.

* \param session The handle identifying the session to add the camera to.

* \param camera A handle to the camera to be added. It is the responsibility of the client code to configure it.

* \param data Allocated memory to write the data to.

* \param noOfFrames The number of frames of data that will be requested from the camera.

```

*/
MS_DECL_SPEC void addCamera(SESSION_HANDLE session, PF32_HANDLE camera,
uint8_t * data, unsigned int noOfFrames);

```

/** \brief Use this method for third party languages that cannot cast pointers. A wrapper that casts the 16 bit pointer to 8 bit and calls addCamera(uint8_t * data)

* \param session The handle identifying the session to add the camera to.

* \param camera A handle to the camera to be added. It is the responsibility of the client code to configure it.

* \param data Allocated memory to write the data to.

* \param noOfFrames The number of frames of data that will be requested from the camera.

```

*/
MS_DECL_SPEC void addCamera_short(SESSION_HANDLE session, PF32_HANDLE camera,
uint16_t * data, unsigned int noOfFrames);

```

/** \brief This method will spawn a thread for each camera calling getNextFrames(). The threads are destroyed afterwards.

The client code can call this method multiple times unless destroySession() is called.

The data read from the camera will be written to the memory pointed to by the data pointer passed to addCamera()

```
* \param session The handle identifying the session to be run.  
* \param buffered When false, an unbuffered read is performed (live data is captured directly from the camera.) When true, a buffered read is performed (data is returned from a software circular buffer, which is filled by a background thread.)  
* \param performInitialPurge If true then the circular buffer (only for buffered reads) and the firmware FIFO will be purged before reading commences.
```

```
*/  
MS_DECL_SPEC void executeSession(SESSION_HANDLE session, bool buffered, bool performInitialPurge);
```

```
/** \brief Call this when you no longer want to use the session.  
* \param session The handle identifying the session to be destroyed.  
*/  
MS_DECL_SPEC void destroySession(SESSION_HANDLE session);
```

/** \brief Method to read and histogram sensor data when the camera is using firmware that returns 8-bit data.

```
* \param pf32 Handle to the PF32 object to be read from.  
* \param data Pointer to buffer in which to write the generated histogram data. The buffer must be at least \ref getNoOfTDCCodes * \ref getEnabledNoOfPixels uint8 bytes.  
* \param noOfSeconds Number of seconds of sensor data to be captured and histogrammed.  
* \return Boolean value indicating success or failure.  
*/  
MS_DECL_SPEC bool getHistogram(PF32_HANDLE pf32, uint16_t *data, double noOfSeconds);
```

```
/** \brief DEPRECATED. Method to read and histogram sensor data.  
* \param pf32 Handle to the PF32 object to be read from.  
* \param data Pointer to buffer in which to write the generated histogram data. The buffer must be at least \ref getNoOfTDCCodes * \ref getEnabledNoOfPixels uint16 words.  
* \param noOfSeconds Number of seconds of sensor data to be captured and histogrammed.  
* \return Boolean value indicating success or failure.  
*/  
MS_DECL_SPEC bool getHistogram_short(PF32_HANDLE pf32, uint16_t *data, double noOfSeconds);
```

/** \brief DEPRECATED. Method to read and histogram sensor data when the camera is using firmware that returns 8-bit data.

* \param pf32 Handle to the PF32 object to be read from.

* \param data Pointer to buffer in which to write the generated histogram data. The buffer must be at least \ref getNoOfTDCCodes * \ref getEnabledNoOfPixels uint8 bytes.

* \param noOfSeconds Number of seconds of sensor data to be captured and histogrammed.

* \return Boolean value indicating success or failure.

*/

MS_DECL_SPEC bool getHistogram_char(PF32_HANDLE pf32, uint8_t *data, double noOfSeconds);

/**

* @brief Set how many frames are used to create a single histogram.

* @return How many frames are used for histogramming. Value is read from the hardware.

*/

MS_DECL_SPEC unsigned int getNoOfFramesToHistogram(PF32_HANDLE handle);

/**

* @brief Set how many frames are used to create a single histogram.

* @param noOfFrames: How many frames are used for histogramming.

*/

MS_DECL_SPEC void setNoOfFramesToHistogram(PF32_HANDLE handle, unsigned int noOfFrames);

/**

* @brief Set how many bins make up a single histogram.

* @param noOfDivisions: How many times should the maximum number of bins be divided by 2. This will be used as the number of bins in the histogram.

*/

MS_DECL_SPEC void setNoOfBinsInHistogram(PF32_HANDLE handle, unsigned int noOfDivisions);

/**

* @brief Get how many bins make up a single histogram.

* @return How many bins are used in the histogram. This is the actual number of bins rather than the number of divisions passed to the setter. Value is read from the hardware.

```

*/
MS_DECL_SPEC unsigned int getNoOfBinsInHistogram(PF32_HANDLE handle);

/**
 * @brief Read a single histogram from the module.
 * @param data Pointer to memory to write the data to. The user is responsible for
pre-allocating sufficient memory for it. Call getNoOfBytesPerHistogram() to find the number of
bytes required.
 */
MS_DECL_SPEC bool getHistogramFromFirmware(PF32_HANDLE handle, uint8_t * data);

/**
 * @brief The size of each histogram in bytes.
 * @retval Number of bytes per frame.
 */
MS_DECL_SPEC unsigned int getNoOfBytesPerHistogram(PF32_HANDLE handle, bool
fromFirmware);

///
/** \brief Returns the number of frames in the circular buffer. May actually be larger
depending on how multiple is set.
 * \param pf32 Handle to the PF32 object to be queried.
 * \return Number of frames stored by the circular buffer
 */
MS_DECL_SPEC unsigned int getNoOfFramesInBuffer(PF32_HANDLE pf32);

/** \brief Sets the number of frames in the circular buffer.
In practise the buffer can be several times larger depending on if multiple is
greater than 1.
Either way noOfFrames is the size of the block that can be read out / written
in one go.
 * \param pf32 Handle to the PF32 object to be queried.
 * \param noOfFrames Number of frames stored by the circular buffer
 */
MS_DECL_SPEC void setNoOfFramesInBuffer(PF32_HANDLE pf32, unsigned int
noOfFrames);

/** \brief Returns the multiple of number of frames in the circular buffer.
 * \param pf32 Handle to the PF32 object to be queried.
 * \return Number of frames stored by the circular buffer

```

```

*/
MS_DECL_SPEC unsigned int getMultipleOfBuffer(PF32_HANDLE pf32);

/** \brief Sets the overall multiple of number of frames in the circular buffer.
    The default is 1 which means that if the data / space is available, up to the
    entirety of the buffer will be read out / written to each time it blocks.
    Setting the multiple to 2 for example means that only half the buffer can be
    read out / written to in one go.
    * \param pf32 Handle to the PF32 object to be queried.
    * \param multiple Multiple of number of frames stored by the circular buffer
*/
MS_DECL_SPEC void setMultipleOfBuffer(PF32_HANDLE pf32, unsigned int multiple);

/** \brief Query the model number of the attached camera.
    * \param pf32 Handle to the PF32 object to be queried.
    * \param buffer Pointer to a char buffer in which to write the camera model number
    string. Buffer must be at least \ref MAX_MODEL_NUMBER_LENGTH + 1 bytes long.
*/
MS_DECL_SPEC void getModelNumber(PF32_HANDLE pf32, char * buffer);

/** \brief Query the serial number of the attached camera.
    * \param pf32 Handle to the PF32 object to be queried.
    * \param buffer Pointer to a char buffer in which to write the camera serial number
    string. Buffer must be at least \ref MAX_SERIAL_NUMBER_LENGTH + 1 bytes long.
*/
MS_DECL_SPEC void getSerialNumber(PF32_HANDLE pf32, char * buffer);

/** \brief Allows the contents of the buffer to be discarded and new frames fetched.
    * \param pf32 Handle to the PF32 object to be updated.
*/
MS_DECL_SPEC void purgeBulkFrameBuffer(PF32_HANDLE pf32);

/** \brief Has the camera being loaded with custom firmware which tells it to return 8-bit
    values rather than 16-bit words that contain 10-bit values.
    * This allows the camera to stream faster at the cost of less information per pixel.
    * \param pf32 Handle to the PF32 object to be updated.
    * \return Returns 1, 2, 4, 8 or 16. This value is determined by the firmware file.
*/
MS_DECL_SPEC unsigned int getBitMode(PF32_HANDLE pf32);

```

```

/// @}

////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
/// \name Misc sensor conf methods
/// @{

/** \brief Set the enable status of the SPADs.
 * \param pf32 Handle to the PF32 object to be updated.
 * \param SPAD_en SPAD enable status (true = active, false = disabled).
 */
MS_DECL_SPEC void setSPADEnable(PF32_HANDLE pf32, bool SPAD_en);

/** \brief Get the current enable status of the SPADs.
 * \param pf32 Handle to the PF32 object to be updated.
 * \return Current SPAD enable status (true = active, false = disabled).
 */
MS_DECL_SPEC bool getSPADEnable(PF32_HANDLE pf32);

/** \brief Set the required data source for streaming.
 * \param pf32 Handle to the PF32 object to be updated.
 * \param source \ref PF32_data_source enum specifying the required data type to be
streamed.
 */
MS_DECL_SPEC void setDataSource(PF32_HANDLE pf32, PF32_data_source source);

/** \brief Query the current streaming data source.
 * \param pf32 Handle to the PF32 object to be updated.
 * \return \ref PF32_data_source enum specifying the currently selected data type to
be streamed.
 */
MS_DECL_SPEC PF32_data_source getDataSource(PF32_HANDLE pf32);

/** \brief Enable generation of internal TCSPC stop signal, useful in camera as master
mode.
 * \param pf32 Handle to the PF32 object to be updated.
 * \param EXTSTOP_enable Boolean to enable/disable generation of the internal TCSPC
stop signal.
 */
MS_DECL_SPEC void setEXTSTOPEnable(PF32_HANDLE pf32, bool EXTSTOP_enable);

/** \brief Query the generation of internal TCSPC stop signal.
 * \param pf32 Handle to the PF32 object to be updated.
 * \return Boolean value indicating the current status of the internal TCSPC stop signal.
 */

```

```

MS_DECL_SPEC bool getEXTSTOPEnable(PF32_HANDLE pf32);

/** \brief Set the number of electrical test pulses to be generated per frame for use in
test pulse counting mode.
* \param pf32 Handle to the PF32 object to be updated.
* \param testPulseCount Number of electrical test pulses to be generated per frame
for use in test pulse counting mode.
*/
MS_DECL_SPEC void setTestPulseCount(PF32_HANDLE pf32, int testPulseCount);

/** \brief Query the number of electrical test pulses to be generated per frame for use in
test pulse counting mode.
* \param pf32 Handle to the PF32 object to be updated.
* \return Number of electrical test pulses to be generated per frame for use in test
pulse counting mode.
*/
MS_DECL_SPEC int getTestPulseCount(PF32_HANDLE pf32);

/** \brief Specifies the position of the electrical test pulse signal with respect to the start
of the frame.
* \param pf32 Handle to the PF32 object to be updated.
* \param testStartDelay Delay in 12.5ns clock periods from frame start to first
electrical test pulse.
*/
MS_DECL_SPEC void setTestStartDelay(PF32_HANDLE pf32, int testStartDelay);

/** \brief Queries the position of the electrical test pulse signal with respect to the start
of the frame.
* \param pf32 Handle to the PF32 object to be updated.
* \return Delay in 12.5ns clock periods from frame start to first electrical test pulse.
*/
MS_DECL_SPEC int getTestStartDelay(PF32_HANDLE pf32);

/** \brief Specifies the position of the internal TCSPC stop signal with respect to the start
of the frame.
* \param pf32 Handle to the PF32 object to be updated.
* \param EXTSTOP_delay Delay in 12.5ns clock periods from frame start to first
internal TCSPC stop signal pulse.
*/
MS_DECL_SPEC void setEXTSTOPDelay(PF32_HANDLE pf32, int EXTSTOP_delay);

/** \brief Enable/disable the camera's shutter output signal.
* \param pf32 Handle to the PF32 object to be updated.
* \param shutterOutState State of the camera's shutter output signal.

```

```

*/
MS_DECL_SPEC void setShutterOutState(PF32_HANDLE pf32, bool shutterOutState);

/** \brief Query the state of the camera's shutter output signal.
 *  \param pf32 Handle to the PF32 object to be updated.
 *  \return State of the camera's shutter output signal.
 */
MS_DECL_SPEC bool getShutterOutState(PF32_HANDLE pf32);
/// @}

////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
/// \name Exposure controls
/// @{

/** \brief Queries the number of clock cycles per line of pixel data.
 *  \param pf32 Handle to the PF32 object to be updated.
 *  \return The number of clock cycles per line of pixel data.
 */
MS_DECL_SPEC int  getBitsPerLine(PF32_HANDLE pf32);

/** \brief Queries the number of lines (rows) of data to be read from each half of the
array per frame.
 *  \param pf32 Handle to the PF32 object to be updated.
 *  \return The number of lines (rows) of data to be read from each half of the array per
frame.
 */
MS_DECL_SPEC int  getLinesPerFrame(PF32_HANDLE pf32);

/** \brief Specifies the number of sensor frames to be summed together in the camera
firmware.
 *
 *      Total exposure time per frame read by software is therefore the sensor
exposure time multiplied by the number of frames to sum in firmware.
 *
 *      Firmware summation supports values up to 16 bit (65,535).
 *  \param pf32 Handle to the PF32 object to be updated.
 *  \param framesToSum The number of sensor frames to be summed together in the
camera firmware.
 */
MS_DECL_SPEC void setFramesToSum(PF32_HANDLE pf32, int framesToSum);

/** \brief Queries the number of sensor frames to be summed together in the camera
firmware.
 *
 *      Total exposure time per frame read by software is therefore the sensor
exposure time multiplied by the number of frames to sum in firmware.
 *
 *      Firmware summation supports values up to 16 bit (65,535).

```



```

    * \param pf32 Handle to the PF32 object to be updated.
    * \return The number of sensor frames to be summed together in the camera
firmware
    */
MS_DECL_SPEC int getFramesToSum(PF32_HANDLE pf32);

/** \brief Set the sensor exposure time for a single frame. The sensor's readout frame
rate is the reciprocal of exposure time.
    * \param pf32 Handle to the PF32 object to be updated.
    * \param exposure Exposure time, expressed as a double, in microseconds.
    */
MS_DECL_SPEC void setExposure_us(PF32_HANDLE pf32, double exposure);

/** \brief Set the sensor exposure time for a single frame.
    * Call this method instead of setExposure_us(exposure) if you want to effectively
reduce the optical exposure
    * further by extending the duration of the pixel reset.
    * \param pf32 Handle to the PF32 object to be updated.
    * \param framePeriod Used same way as setExposure_us(exposure), expressed as a
double, in microseconds.
    * \param shutterSpeed This reduces how much of the frame period is disabled,
expressed as a double, in microseconds.
    */
MS_DECL_SPEC void setFramePeriodAndOpticalExposure_us(PF32_HANDLE pf32, double
framePeriod, double opticalExposure);

/** \brief Query the sensor exposure time for a single frame. The sensor's readout frame
rate is the reciprocal of exposure time.
    * \param pf32 Handle to the PF32 object to be updated.
    * \return Exposure time, expressed as a double, in microseconds.
    */
MS_DECL_SPEC double getExposure_us(PF32_HANDLE pf32);

/** \brief Allows manual configuration of the low-level timing of the sensor's frame
readout, hence exposure time.
    * Note that the \ref setExposure_us method is the recommended way of
setting exposure time.
    * Please consult Photon Force support before using this method.
    * Sensor exposure time = bitsPerLine * linesPerFrame * sensor clock period.
    * \param pf32 Handle to the PF32 object to be updated.
    * \param bitsPerLine The number of clock cycles per line of pixel data.
    * \param linesPerFrame The number of lines (rows) of data to be read from each half
of the array per frame.

```

```

*/
MS_DECL_SPEC void setLineTiming(PF32_HANDLE pf32, int bitsPerLine, int
linesPerFrame);

/** \brief Allows manual configuration of the low-level timing of the sensor's frame
readout, hence exposure time.
*      Note that the \ref setExposure_us method is the recommended way of
setting exposure time.
*      Please consult Photon Force support before using this method.
*      \param pf32 Handle to the PF32 object to be updated.
*      \param bitsPerLine The number of clock cycles per line of pixel data.
*/
MS_DECL_SPEC void setBitsPerLine(PF32_HANDLE pf32, int bitsPerLine);

/** \brief Query the sensor's readout clock frequency. This is useful when modifying line
timing settings using the low level \ref getBitsPerLine, \ref getLinesPerFrame, or \ref
setLineTiming methods.
*      \param pf32 Handle to the PF32 object to be updated.
*      \return Current sensor readout clock frequency, expressed as an integer, in Hz.
*/
MS_DECL_SPEC int getSensorClk_Hz(PF32_HANDLE pf32);

/** \brief Query the current sync input frequency.
*      \param pf32 Handle to the PF32 object to be updated.
*      \return Current sync input frequency, expressed as an integer, in Hz.
*/
MS_DECL_SPEC int getSync_Hz(PF32_HANDLE pf32);

/** \brief Query the current sync input duty ratio. This is useful to check that the signal
polarity is as expected.
*      \param pf32 Handle to the PF32 object to be updated.
*      \return Current sync input signal duty ratio, expressed as a double, from 0 to 1.
*/
MS_DECL_SPEC double getSyncDutyRatio(PF32_HANDLE pf32);
/// @}

////////////////////////////////////
/// \name Callbacks
/// @{
/** \brief Adds callback routine to allow output messages from the driver to be displayed
by third party code.
*      \param callback Function pointer to a callback routine. Callback method takes three

```

parameters:

- * int verboseLevel (corresponding with enum LOGLEVEL), a pointer to a string describing what part of the code the message pertains to and its length, and a pointer to the actual message and its length.

*/

```
MS_DECL_SPEC void setLogCallback(void(*callback)(int, const char *, int, const char *,
int));
```

/** \brief Adds callback routine that gets called when the status of the camera changes.

- * \param pf32 Handle to the PF32 object to be updated.

- * \param callback Function pointer that accepts one parameter that gets called when camera status changes. The parameter corresponds to PF32_conn_status

*/

```
MS_DECL_SPEC void setStatusCallback(PF32_HANDLE pf32, void(*callback)(int));
```

/// @}

//

/// \name Region of interest

/// @{

/** \brief Get which pixels are being returned from the camera. Array of bools for columns and rows will be populated by method, and also for rows.

- * Only the pixels intersected by both a row and column marked as true will be returned.

- * Default is for all pixels to be turned on.

- * NOTE: get/set RegionsOfInterest() is currently work in progress and has not yet been tested.

- * \param pf32 Handle to the PF32 object to query.

- * \param columns An array of 32 bools for array width

- * \param rows An array of 32 bools for array height

*/

```
MS_DECL_SPEC void getRegionsOfInterest(PF32_HANDLE pf32, bool * columns, bool *
rows);
```

/** \brief Set which pixels should be returned from the camera by passing in an array of bools for columns, and also for rows.

- * Only the pixels intersected by both a row and column marked as true will be returned.

- * Default is for all pixels to be turned on.

- * NOTE: get/set RegionsOfInterest() is currently work in progress and has not yet been tested.

- * \param pf32 Handle to the PF32 object to be updated.

- * \param columns An array of 32 bools for array width

```

    * \param rows An array of 32 bools for array height
    */
    MS_DECL_SPEC void setRegionsOfInterest(PF32_HANDLE pf32, const bool * columns,
const bool * rows);
    /// @}

/////////////////////////////////////////////////////////////////
    /// \name Opal Kelly functions - deprecated, available for backwards compatibility.
    /// @{
    MS_DECL_SPEC int GetDeviceMajorVersion(PF32_HANDLE pf32);
    MS_DECL_SPEC int GetDeviceMinorVersion(PF32_HANDLE pf32);
    MS_DECL_SPEC void GetSerialNumber(PF32_HANDLE pf32, char *buf);
    MS_DECL_SPEC void SetTimeout(PF32_HANDLE pf32, int timeout);

    MS_DECL_SPEC void UpdateWireIns(PF32_HANDLE pf32);
    MS_DECL_SPEC int GetWireInValue(PF32_HANDLE pf32, int epAddr, unsigned int *val);
    MS_DECL_SPEC int SetWireInValue(PF32_HANDLE pf32, int ep, unsigned long val,
unsigned long mask);
    MS_DECL_SPEC void UpdateWireOuts(PF32_HANDLE pf32);
    MS_DECL_SPEC unsigned long GetWireOutValue(PF32_HANDLE pf32, int epAddr);
    MS_DECL_SPEC int ActivateTriggerIn(PF32_HANDLE pf32, int epAddr, int bit);
    MS_DECL_SPEC void UpdateTriggerOuts(PF32_HANDLE pf32);
    MS_DECL_SPEC int IsTriggered(PF32_HANDLE pf32, int epAddr, unsigned long mask);

    MS_DECL_SPEC long ReadFromPipeOut(PF32_HANDLE pf32, int epAddr, long length,
unsigned char *data);
    MS_DECL_SPEC long ReadFromBlockPipeOut(PF32_HANDLE pf32, int epAddr, int
blockSize, long length, unsigned char *data);
    /// @}

/////////////////////////////////////////////////////////////////
    /// \name Positional data methods
    /// \brief These methods are used for scanner support whereby a footer is sent after each
frame containing positional data.
    ///      Enabling footers will reduce throughput from camera to the PC as more data
per frame needs to be returned.
    ///      Each footer is 16 bytes long and consists of:
    ///      Two 16 bit markers with the values 0x1111 and 0x2222.
    ///      32 bit Frame count.
    ///      16 bit scan point marker.
    ///      X, Y and Z 16 bit values.

```

```
/** \brief Switches on / off scanner support if this feature is available otherwise it returns after outputting an error message.
```

```
 *           If switched on then the camera will return a 16 byte footer will be output by the camera after each frame.
```

```
 * \param pf32 Handle to the PF32 object to be updated.
```

```
 * \param enableFooters boolean parameter whether to switch scanner support on or off.
```

```
 */
```

```
MS_DECL_SPEC void setEnableFooters(PF32_HANDLE pf32, bool enableFooters);
```

```
/** \brief Gets whether scanner support has been turned on. If so then a 16 byte footer containing positional data will be output by the camera and driver after each frame.
```

```
 * \param pf32 Handle to the PF32 object to be updated.
```

```
 * \return Whether a footer will be returned after each frame.
```

```
 */
```

```
MS_DECL_SPEC bool getEnableFooters(PF32_HANDLE pf32);
```

```
/** \brief Deprecated. Duplicate of setEnableFooters().
```

```
           Switches on / off scanner support if this feature is available otherwise it returns after outputting an error message.
```

```
 *           If switched on then the camera will return a 16 byte footer will be output by the camera after each frame.
```

```
 * \param pf32 Handle to the PF32 object to be updated.
```

```
 * \param enablePositionalData boolean parameter whether to switch scanner support on or off.
```

```
 */
```

```
MS_DECL_SPEC void setEnablePositionalData(PF32_HANDLE pf32, bool enablePositionalData);
```

```
/** \brief Deprecated. Duplicate of getEnableFooters()
```

```
           Gets whether scanner support has been turned on. If so then a 16 byte footer containing positional data will be output by the camera and driver after each frame.
```

```
 * \param pf32 Handle to the PF32 object to be updated.
```

```
 * \return Whether a footer will be returned after each frame.
```

```
 */
```

```
MS_DECL_SPEC bool getEnablePositionalData(PF32_HANDLE pf32);
```

```
/** \brief Helper method to make iterating through raw data already returned by the
```

camera easier when it contains positional data in footers for scanner support.

- * Copies the frame and the footer contained in the first parameter to the 3rd and 4th parameter respectively

- * This means that the client code does not have to concern itself with pointer arithmetic or the format of the footer but it does in

- * involve copying so is expensive to use.

- * \param pf32 Handle to the PF32 object to be updated.

- * \param data The beginning of the array containing the raw frame data and footers.

- * \param whichFrame The frame and its footer to be extracted, counted from the beginning of the array pointed to by data

- * \param frameData Pointer to an array to copy the frame data to.

- * \param positionalData Pointer to an array to copy the positional data to. This will contain the frame count, X, Y and Z co-ordinates, each a 32 bit value.

- * \param enabledHeight This parameter specifies how many rows appear in each frame when regions of interest are set.

- */

```
MS_DECL_SPEC void iteratePositionalData_short(PF32_HANDLE pf32, uint16_t * data,
unsigned int whichFrame, uint16_t * frameData, uint32_t * positionalData, unsigned int
enabledHeight);
```

```
/// @}
```

```
////////////////////////////////////
```

```
/// \name Cooling methods
```

```
/// \brief Temperature controller methods, only applicable to Mk4 cooled camera. The
methods will have no effect on non-cooled cameras.
```

```
/// The driver can be given a target temperature, but cooling needs to be explicitly
switched on or off.
```

```
/** \brief Set the target temperature the camera will attempt to cool the sensor to. Use
setEnabledCooling(true) to start cooling.
```

- * Only works with cameras which feature Peltier sensor cooling.

- * \param pf32 Handle to the PF32 object to be updated.

- * \param setpoint_K Double value specifying required temperature set point in Kelvin.

- */

```
MS_DECL_SPEC void setTargetTemp(PF32_HANDLE pf32, double setpoint_K);
```

```
/** \brief Get the current temperature of the camera.
```

- * Only works with cameras which feature Peltier sensor cooling.

- * \param pf32 Handle to the PF32 object to be updated.

- * \return Double value specifying the current (actual) sensor temperature in Kelvin.

- */

```
MS_DECL_SPEC double getActualTemp(PF32_HANDLE pf32);
```

```

/** \brief Tell the camera to start or stop cooling.
 *   Only works with cameras which feature Peltier sensor cooling.
 *   \param pf32 Handle to the PF32 object to be updated.
 *   \param enableCooling Boolean value specifying whether the camera Peltier cooling
should be enabled. If called on a camera without this ability it will return false.
 *   \return Boolean value indicating success or failure.
 */
MS_DECL_SPEC bool setEnableCooling(PF32_HANDLE pf32, bool enableCooling);

/** \brief Gets whether the camera is currently cooling itself.
 *   Only works with cameras which feature Peltier sensor cooling.
 *   \param pf32 Handle to the PF32 object to be updated.
 *   \return Returns boolean value specifying whether the camera is currently cooling
itself. If called on a camera without this ability it will return false.
 */
MS_DECL_SPEC bool getEnableCooling(PF32_HANDLE pf32);

/** \brief Get the current temperature of the camera board if it's a USBC version 2 model.
 *   \param pf32 Handle to the PF32 object to be updated.
 *   \return Double value specifying the current board temperature in degrees C.
 */
MS_DECL_SPEC double getBoardTemp(PF32_HANDLE pf32);

/// @}

////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
/// \name Sync methods
/// \brief These methods are used for the Sync SMA input.
///

/** \brief Set whether the Sync SMA has been inverted for a positive or negative pulse.
 *   Can only be used by cameras which are PF32_USBC version 2.
 *   \param pf32 Handle to the PF32 object to be updated.
 *   \param positive Value specifying whether Sync SMA is positive. Negative means it is
inverted.
 *   This value is non-volatile and is kept even after the camera is
switched off and reinitialised.
 */
MS_DECL_SPEC void setSyncPolarity(PF32_HANDLE hnd, bool positive);

```

```
/** \brief Get the threshold of that the Sync SMA input is compared to.  
 *  
 * Can only be used by cameras which are PF32_USBC version 2.  
 * \param pf32 Handle to the PF32 object to be queried.  
 * \return Bool Value specifying the threshold Sync SMA is compared to.  
 */
```

```
MS_DECL_SPEC int getSyncThreshold(PF32_HANDLE hnd);
```

```
/** \brief Set the threshold of that the Sync SMA input is compared to.  
 *  
 * Can only be used by cameras which are PF32_USBC version 2.  
 * \param pf32 Handle to the PF32 object to be updated.  
 * \param threshold Value specifying the threshold Sync SMA is compared to.  
 */
```

```
MS_DECL_SPEC void setSyncThreshold(PF32_HANDLE hnd, int threshold);
```

```
/// @}
```